```
import os
import mlflow
from mlflow.store.artifact.models_artifact_repo import ModelsArtifactRepository
from mlia_utils.cv_clf_funcs import select_best_model

experiment_path = f"/Users/{current_user}/intel-clf-training_action"
local_path = select_best_model(experiment_path)

requirements_path = os.path.join(local_path, "requirements.txt")
if not os.path.exists(requirements_path):
  dbutils.fs.put("file:" + requirements_path, "", True)

device = "cuda" if torch.cuda.is_available() else "cpu"
loaded_model = torch.load(local_path+"/data/model.pth", map_location=torch.device(device))

wrapper = CVModelWrapper(loaded_model)
```

Figure 7.32 – Load our model from the existing MLFlow experiment

Let's test whether our wrapper is functioning as expected. To do so, we encode a few images (as the model serving accepts strings and cannot yet accept images and convert them) and save them into a Pandas DataFrame. We then use our model wrapper to get a prediction.

```
import base64
import pandas as pd

images = spark.read.format("delta").load(val_delta_path).take(25)

b64image1 = base64.b64encode(images[0]["content"]).decode("ascii")
b64image2 = base64.b64encode(images[1]["content"]).decode("ascii")
b64image3 = base64.b64encode(images[3]["content"]).decode("ascii")
b64image4 = base64.b64encode(images[4]["content"]).decode("ascii")
b64image24 = base64.b64encode(images[24]["content"]).decode("ascii")

df_input = pd.DataFrame(
    [b64image1, b64image2, b64image3, b64image4, b64image24], columns=["data"])

df = wrapper.predict("", df_input)
display(df)
```

Figure 7.33 – Using our model wrapper to create predictions on a few images